🏆 码蹄杯国赛速查手册

> 基于 161 道 LeetCode 题目的竞赛知识点提炼
> 涵盖：核心知识点 · 代码模板 · 解题思路 · 易错陷阱

目录

1. 数组与遍历
2. 双指针
3. 滑动窗口
4. 二分查找
5. 前缀和与差分
6. 排序与贪心
7. 栈
8. 队列与双端队列
9. 字符串
10. 位运算与数学
11. 动态规划
12. 图论——BFS / Dijkstra
13. 二叉树与递归
14. 矩阵
15. 状态压缩
16. 堆与优先队列
17. 并查集
18. 思维技巧汇总
19. 树状数组与线段树
20. 拓扑排序与最小生成树
21. 数位 DP
22. 灵神模板补充——高性能 IO 与杂项

一、数组与遍历

1.1 原地删除（允许 k 次重复）

模板	说明
`slow = k; for (fast = k; fast < n; fast++)`	比较 `nums[fast]` 与 `nums[slow-k]`
if (nums[fast] != nums[slow-k]) → nums[slow++] = nums[fast]	不相等则保留

思路：慢指针指向「已处理的正确位置」，快指针扫描全部元素。
易错点：`slow` 从 k 开始（前 k 个一定合法），不要从 0 开始。

1.2 除自身以外乘积

```
vector<int> ans(n, 1);
for (int i = 1; i < n; i++) ans[i] = ans[i-1] * nums[i-1]; // 左前缀
int right = 1;
for (int i = n-1; i >= 0; i--) { ans[i] *= right; right *= nums[i]; }
```

易错点：先乘 `right` 再更新 `right`，顺序别反。

1.3 摩尔投票（求众数）

```
int x = 0, vote = 0;
for (int num : nums) {
    if (vote == 0) x = num;
    vote += (num == x) ? 1 : -1;
}
return x;
```

思路：不同两两抵消，相同累加，剩下的就是众数。
扩展：求 > ⌊n/3⌋ 的元素 → 维护两个候选，两个计数器。

二、双指针

2.1 相向双指针（两数之和）

```
int l = 0, r = n - 1;
while (l < r) {
    int sum = nums[l] + nums[r];
    if (sum == target) return {l, r};
    sum < target ? l++ : r--;
}
```

适用条件：数组有序。

2.2 同向双指针（推多米诺、长按键入）

两个指针分别扫描两个序列，根据条件决定哪个前进。

易错点：边界条件处理——确保指针不越界。

2.3 对比表

题型	指针移动	典型题	易错点
两数之和	相向	167	需要有序
两端删除	相向收缩	1750	跳过连续相同字符
下标对	同向不同步	1855	循环条件
字符串匹配	同向	925	typed 可能有重复
供暖器	同向	475	排序后逐个匹配
差值约束	同向扩展	排序+双指针	左边界移动时机

三、滑动窗口

3.1 定长窗口骨架

```
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += nums[i];          // 入窗
    if (i >= k) sum -= nums[i - k];  // 出窗
    if (i >= k - 1) ans = max(ans, sum); // 更新答案
}
```

易错点：出窗时机是在更新答案之前还是之后，不要搞混。

3.2 不定长窗口骨架

```
int l = 0;
for (int r = 0; r < n; r++) {
    // 扩展右边界，更新状态
    while (/* 窗口违反约束 */) {
        // 收缩左边界，更新状态
        l++;
    }
    ans = max(ans, r - l + 1);  // 或 min 等
}
```

思路：右指针扩张直至违反条件，左指针收缩直至满足条件。
易错点：收缩时注意对应状态要同步还原。

3.3 常见约束条件速查

题目	约束条件	收缩条件
最短和 ⩾ target	sum < target	`while (sum >= target)`
最多 k 个 0	zeros > k	`while (zeros > k)`
至多 2 种水果	types > 2	`while (types > 2)`
最值差 ⩽ limit	max-min > limit	`while (max-min > limit)`
含所有 3 字符	三种字符都出现	`while (三种都出现)`

3.4 枚举右，维护左（滑动窗口变体）

```
// 1014. 观光景点评分：values[i] + i + values[j] - j
int ans = 0, max_i = values[0];  // max_i = values[i] + i
for (int j = 1; j < n; j++) {
    ans = max(ans, max_i + values[j] - j);
    max_i = max(max_i, values[j] + j);
}
```

思路：将需要枚举两个变量的式子拆解为 `f(i) + g(j)`，枚举 j 的同时维护 f_i 的最大值。

四、二分查找

4.1 标准模板

```
int l = 0, r = n;  // 左闭右开
while (l < r) {
    int mid = l + (r - l) / 2;
    if (check(mid)) r = mid;
    else l = mid + 1;
}
return l;
```

变体	条件	返回值
第一个 ⩾ target	`nums[mid] < target` → l=mid+1	l
第一个 > target	`nums[mid] <= target` → l=mid+1	l
最后一个 ⩽ target	先找第一个 > target，再 -1	l-1

4.2 值域二分（答案二分）

适用场景：求「最大值最小化」或「最小值最大化」，且答案在值域上单调。

```
int l = min_possible, r = max_possible;
while (l < r) {
    int mid = l + (r - l) / 2;
    if (check(mid)) r = mid;  // 根据单调性决定方向
    else l = mid + 1;
}
```

易错点：`check()` 函数的正确定义——是判断「可行」还是「不可行」。

4.3 高频 Check 函数一览

题	二分对象	check 逻辑
875 吃香蕉	速度 k	模拟小时数 ⩽ H
1011 运包裹	运载量	模拟天数 ⩽ D
1283 最小除数	除数	sum$ceil(v/div)$ ⩽ threshold
2226 分糖果	每人颗数	���分到的人数 ⩾ 总人数
378 矩阵第k小	数值	矩阵中 ⩽ mid 的个数 ⩾ k

五、前缀和与差分

5.1 前缀和

```
vector<int> pre(n + 1, 0);
for (int i = 1; i <= n; i++) pre[i] = pre[i-1] + nums[i-1];
// 区间 [l, r] 和 = pre[r+1] - pre[l]
```

5.2 一维差分

```
diff[l] += val;
diff[r+1] -= val;
// 还原
for (int i = 1; i <= n; i++) diff[i] += diff[i-1];
```

思路：区间加减操作 → O1 修改，On 还原。

5.3 二维差分

```
diff[r1+1][c1+1]++;
diff[r1+1][c2+2]--;
diff[r2+2][c1+1]--;
diff[r2+2][c2+2]++;

// 二维前缀和还原
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++)
        diff[i][j] += diff[i-1][j] + diff[i][j-1] - diff[i-1][j-1];
```

5.4 前缀和 + 哈希（和为目标值的子数组计数）

```
unordered_map<int, int> mp; mp[0] = 1;
int sum = 0, ans = 0;
for (int x : nums) {
    sum += x;
    ans += mp[sum - k];  // 看前面有多少个前缀和 = sum - k
    mp[sum]++;
}
```

易错点：`mp[0] = 1` 表示空前缀，别忘了初始化。

六、排序与贪心

6.1 区间贪心

452. 最少箭射气球——按右端点排序

```
sort(points, [](auto& a, auto& b) { return a[1] < b[1]; });
int ans = 1, pre = points[0][1];
for (int i = 1; i < n; i++) {
    if (points[i][0] > pre) { ans++; pre = points[i][1]; }
}
```

思路：每次选「最早结束」的区间，在其右端点放箭。
易错点：是按右端点还是左端点排序？区间覆盖问题通常按右端点。

6.2 跳跃游戏

45. 跳跃游戏 II——反向贪心

```
int pos = n - 1, ans = 0;
while (pos > 0) {
    for (int i = 0; i < pos; i++) {
        if (i + nums[i] >= pos) { pos = i; ans++; break; }
    }
}
```

或者正向 BFS 风格：

```
int cur = 0, next = 0, ans = 0;
for (int i = 0; i < n - 1; i++) {
    next = max(next, i + nums[i]);
    if (i == cur) { cur = next; ans++; }
}
```

6.3 贪心核心框架

1. 排序：按什么顺序？(右端点 / 左端点 / 值)
2. 选择：每一步选「当前最优」
3. 证明：交换论证 / 反证法

易错点：贪心不成立时立即转 DP，不要死磕。

七、栈

7.1 括号匹配

```
stack<int> stk;
unordered_map<char,char> mp = {{')','('}, {']','['}, {'}','{'}};
for (char c : s) {
    if (mp.count(c)) {
        if (stk.empty() || stk.top() != mp[c]) return false;
        stk.pop();
    } else stk.push(c);
}
return stk.empty();
```

易错点：左括号入栈，右括号检查栈顶是否匹配。别忘了检查 `stk.empty()`。

7.2 相邻消除

```
// 删除相邻相同字母
int top = -1;
for (int i = 0; s[i]; i++) {
    if (top >= 0 && stack[top] == s[i]) top--;
    else stack[++top] = s[i];
}
stack[top+1] = '\0';
```

易错点：`top >= 0` 先判断空栈。

7.3 单调栈

739. 下一个更大元素

```
stack<int> stk;
for (int i = 0; i < n; i++) {
    while (!stk.empty() && nums[i] > nums[stk.top()]) {
        ans[stk.top()] = i - stk.top();
        stk.pop();
    }
    stk.push(i);
}
```

思路：单调递减栈（栈底到栈顶递减），遇到更大元素就出栈。

易错点：
1. 栈中存的是**下标**不是值（方便计算距离）
2. 比较的是 `nums[i]` 与 `nums[stk.top()]`
3. 单调性方向取决于题目——「下一个更大」用递减栈

7.4 单调栈常见题型

题	栈特性	核心思路
739 每日温度	单调递减	下一个更大距离
84 柱状图最大矩形	单调递增	左右第一个更小位置
456 132模式	单调递减+前缀min	维护 3 候选
768 最多排序块	单调递增	块最大值递增
3113 边界最大子数组	单调递减+计数	相等值累计
1776 车队II	单调递减	从右向左处理碰撞时间

八、队列与双端队列

8.1 单调队列（滑动窗口最值）

```
deque<int> dq;
for (int i = 0; i < n; i++) {
    // 维护递减队列（最大值在 front）
    while (!dq.empty() && nums[dq.back()] < nums[i]) dq.pop_back();
    dq.push_back(i);
    // 移除出窗元素
    if (i >= k && dq.front() == i - k) dq.pop_front();
    if (i >= k - 1) ans[i-k+1] = nums[dq.front()];
}
```

思路：front 是最值，back 是新元素入口。维护单调性。
易错点：出窗判断用 `dq.front() == i - k`（精确匹配下标），不是 `<=`。

8.2 双端队列反向模拟

950. 正序显示牌面
- 正向：显示第一张，把下一张放到末尾 → 很难直接模拟
- **反向**：从大到小排序，每次把末尾元素移到开头，再插入新元素

思路：正向操作复杂时，尝试反向推演。

九、字符串

9.1 字符串常用 API

操作	C	C++
长度	`strlen(s)`	`s.size()` / `s.length()`
比较	`strcmp(s1, s2)`	`s1 == s2`
前缀比较	`strncmp(s1, s2, n)`	`s1.substr(0, n) == s2`
子串	手动复制	`s.substr(pos, len)`
替换	手动	`s.replace(pos, len, str)`
删除	手动	`s.erase(pos, len)`
数字转字符串	`sprintf(buf, "%d", n)`	`to_string(n)`
字符串转数字	`atoi(s)` / `strtol`	`stoi(s)` / `stoll(s)`

9.2 前缀匹配（数字转字符串前缀）

3043. 最长公共前缀

```
unordered_set<int> set;
for (int x : arr1) for (; x; x /= 10) set.insert(x);
int ans = 0;
for (int x : arr2) {
    while (!set.contains(x) && x) x /= 10;
    ans = max(ans, x);
}
return ans ? to_string(ans).size() : 0;
```

十、位运算与数学

10.1 位运算速查表

操作	代码	说明
判 2 的幂	`n > 0 && (n & (n-1)) == 0`	去掉最低位 1 后为 0
判奇偶	`x & 1`	1 奇 0 偶
取最低位 1	`x & -x`	lowbit
去掉最低位 1	`x & (x-1)`	—
大小写互换	`c ^ 32`	A↔a
大写判据	`(c >> 5) & 1`	0=大写, 1=小写
字母序号	`c & 31`	A/a→1, Z/z→26
异或周期	`x^0=x, x^x=0`	可用于去重
数 1 个数	`__builtin_popcount(x)`	GCC 内建

10.2 同余分组

1497. 数组对可被 k 整除

```
int cnt[k] = {0};
for (int x : arr) cnt[(x % k + k) % k]++;
if (cnt[0] % 2) return false;
for (int i = 1; i < k; i++) if (cnt[i] != cnt[k-i]) return false;
```

易错点：处理负数取模——`(x % k + k) % k`。

10.3 数根（数字根）

```
digit_root = (n - 1) % 9 + 1;
```

十一、动态规划

11.1 一维 DP 模板

```
vector<int> dp(n + 1, INF / 0);
dp[0] = base;
for (int i = 1; i <= n; i++) {
    for (int j = 0; j < choices; j++) {
        if (valid) dp[i] = min(dp[i], dp[i - cost[j]] + 1);
        // 或 max
    }
}
```

11.2 经典一维 DP 对照表

题	dp i 含义	转移	初始化
322 零钱兑换	组成 i 的最小硬币数	`dp[i-coin]+1`	`dp[0]=0` , 其余 INF
198 打家劫舍	前 i 家最大金额	max(dp[i-1], dp[i-2]+nums[i])	前两家单独处理
279 完全平方数	组成 i 的最小个数	`dp[i-j*j]+1`	`dp[0]=0`
139 单词拆分	前 i 个字符可拆分	dp[j] && dict.contains(s[j:i])	dp[0]=true

11.3 二维 DP 表格法

1143. 最长公共子序列

```
vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
for (int i = 1; i <= m; i++)
    for (int j = 1; j <= n; j++)
        if (text1[i-1] == text2[j-1])
            dp[i][j] = dp[i-1][j-1] + 1;
        else
            dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
```

72. 编辑距离

```
if (word1[i-1] == word2[j-1]) dp[i][j] = dp[i-1][j-1];
else dp[i][j] = min({dp[i-1][j-1], dp[i-1][j], dp[i][j-1]}) + 1;
```

易错点：字符串 DP 下标通常从 1 开始，注意 `text1[i-1]` 与 `text2[j-1]` 的对应关系。

11.4 组合数 vs 排列数（零钱兑换）

```
// 组合数（硬币在外层）—— 不考虑顺序
for (int coin : coins)
    for (int i = coin; i <= amount; i++)
        dp[i] += dp[i - coin];

// 排列数（金额在外层）—— 考虑顺序
for (int i = 1; i <= target; i++)
    for (int coin : coins)
        if (coin <= i) dp[i] += dp[i - coin];
```

易错点：循环层级的顺序决定了是组合还是排列！先遍历物品 = 组合。

11.5 最长递增子序列（LIS）计数

```
vector<int> len(n, 1), cnt(n, 1);
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        if (nums[j] < nums[i]) {
            if (len[j] + 1 > len[i]) { len[i] = len[j] + 1; cnt[i] = cnt[j]; }
            else if (len[j] + 1 == len[i]) cnt[i] += cnt[j];
        }
    }
}
```

十二、图论——BFS / Dijkstra

12.1 BFS 模板（分层扩散）

```
int dirs[4][2] = {{-1,0},{1,0},{0,-1},{0,1}};
queue<pair<int,int>> q;
// 所有源点入队，标记已访问
int ans = 0;
while (!q.empty()) {
    int sz = q.size();
    while (sz--) {
        auto [i, j] = q.front(); q.pop();
        for (auto& d : dirs) {
            int ni = i + d[0], nj = j + d[1];
            if (合法 && grid[ni][nj] == 未访问) {
                grid[ni][nj] = 已访问;
                q.push({ni, nj});
            }
        }
    }
    ans++;
}
```

易错点：

1. 入队时就标记已访问（防止重复入队）
2. 多源 BFS 先把所有源点入队再开始扩散

12.2 Dijkstra（单源最短路）

```
vector<int> dist(n+1, INT_MAX);
priority_queue<pair<int,int>, vector<...>, greater<>> pq;
dist[start] = 0; pq.push({0, start});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;  // 关键：跳过过期数据
    for (auto [v, w] : g[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            pq.push({dist[v], v});
        }
    }
}
```

易错点：

1. `if (d > dist[u]) continue;` —— 必须跳过旧数据
2. 邻接表建图：`g[from].push_back({to, weight})`
3. 距离初始化为 `INT_MAX`

十三、二叉树与递归

13.1 递归比较

```
bool isSame(TreeNode* p, TreeNode* q) {
    if (!p && !q) return true;
    if (!p || !q) return false;
    return p->val == q->val && isSame(p->left, q->left) && isSame(p->right, q->right);
}

bool isSymmetric(TreeNode* root) {
    return check(root->left, root->right);
}
bool check(TreeNode* l, TreeNode* r) {
    if (!l && !r) return true;
    if (!l || !r) return false;
    return l->val == r->val && check(l->left, r->right) && check(l->right, r->left);
}
```

易错点：对称树比较的是 `left->right` vs `right->left`，不是同侧。

十四、矩阵

14.1 方向数组

```
int dirs[4][2] = { {-1,0}, {1,0}, {0,-1}, {0,1} };  // 四方向
int dirs8[8][2] = { {-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},{1,1} };
```

14.2 二维前缀最小值

3148. 两数差最大值——维护左上方最小值

```
for (int i = 1; i < n; i++) {
    for (int j = 1; j < m; j++) {
        min[i][j] = min({min[i-1][j], min[i][j-1], grid[i-1][j-1]});
        ans = max(ans, grid[i][j] - min[i][j]);
    }
}
```

十五、状态压缩

15.1 前缀奇偶状态压缩

1371. 元音偶数次的最长子串

状态：5 位二进制，分别表示 a/e/i/o/u 出现次数的奇偶性

```
int state = 0;  // 00000
int first[32]; memset(first, -1, sizeof(first)); first[0] = 0;
for (int i = 0; i < n; i++) {
    if (s[i] == 'a') state ^= 1;
    else if (s[i] == 'e') state ^= (1 << 1);
    // ... 其他元音
    if (first[state] == -1) first[state] = i + 1;
    else ans = max(ans, i + 1 - first[state]);
}
```

思路：相同状态出现的两个位置之间的子串，每个字符出现次数为偶数。

15.2 位图统计字母

```
int mask[2] = {0};
for (char c : word) mask[c >> 5 & 1] |= 1 << (c & 31);
return __builtin_popcount(mask[0] & mask[1]);
```

分解：
- `c >> 5 & 1`：大写=0，小写=1
- `c & 31`：字母序号 1~26
- `mask[0] & mask[1]`：同时出现的大小写字母

十六、堆与优先队列

16.1 模板

```
priority_queue<int> pq;                       // 最大堆
priority_queue<int, vector<int>, greater<>> pq;  // 最小堆
priority_queue<pair<int,int>> pq;             // pair 最大堆

// 原地堆化（O(1)空间）
make_heap(v.begin(), v.end());
pop_heap(v.begin(), v.end());  // 堆顶到末尾
push_heap(v.begin(), v.end()); // 末尾入堆
```

16.2 TopK 模式

```
// 维护第 K 小：用最大堆存最小的 K 个
priority_queue<int> pq;
for (int x : nums) {
    pq.push(x);
    if (pq.size() > k) pq.pop();
}
return pq.top();  // 第 K 小
```

易错点：
- 第 K **小** → 最大堆（堆顶是 K 个中最大的，即第 K 小）
- 第 K **大** → 最小堆（堆顶是 K 个中最小的，即第 K 大）

十七、并查集

17.1 模板

```
vector<int> p;
int find(int x) {
    if (p[x] != x) p[x] = find(p[x]);  // 路径压缩
    return p[x];
}
void merge(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra != rb) p[ra] = rb;
}
// 初始化
p.resize(n);
for (int i = 0; i < n; i++) p[i] = i;
```

17.2 应用场景

题	用法	关键点
684 删除成环边	遍历边，成环即答案	边按顺序 merge
200 岛屿数量	每块陆地向左/上 merge	二维转一维或 pair find
1319 连通所有节点	统计连通分量	ans = 分量数 - 1
765 情侣牵手	每对情侣为一组	find(l/2) 与 find(r/2) 合并

易错点：
1. 路径压缩：`p[x] = find(p[x])` 不是 `return find(p[x])`
2. 合并时检查是否已是同一集合
3. 二维并查集可用 `pair<int,int>` 或 `i*col+j` 降维

十八、思维技巧汇总

18.1 「枚举右，维护左」

适用于需要枚举两个变量 `(i, j)`，且条件可以分解为 `f(i) + g(j)` 形式的问题。

```
// 通用框架
int best = init_value;
for (int j = 0; j < n; j++) {
    ans = max(ans, best + calc_right(j));  // 用已维护的左值计算
    best = max(best, calc_left(j));        // 更新左值
}
```

典型题：1014 观光景点、2871 有序三元组、1464 最大乘积

18.2 正难则反

正向操作复杂时，尝试反向推演：
- 950 牌面顺序：正向操作 → 反向还原
- 复杂删除/修改操作 → 从后往前处理
- 求最少步数 → 考虑 BFS 逆向

18.3 等式变换

遇到复杂等式，尝试移项：
- `nums[i] + rev(nums[j]) == nums[j] + rev(nums[i])`
- → `nums[i] - rev(nums[i]) == nums[j] - rev(nums[j])`
- 转化为统计差值出现的次数

18.4 差分优化

区间加减操作 → 差分数组 O1 修改
二维区间加减 → 二维差分四角标记

18.5 不变量思维

与其追踪「何时需要合并/处理」，不如维护一个始终满足不变量的数据结构：
- 单调栈：栈内始终单调
- 并查集：每个集合有且仅有一个根
- 滑动窗口：窗口内始终满足约束

十九、树状数组与线段树

19.1 树状数组（Fenwick Tree）

功能：单点更新 + 区间求和，O$logn$

```cpp
struct Fenwick {
    int n;
    vector<int> tree;
    Fenwick(int n_) : n(n_), tree(n_ + 1, 0) {}

    void add(int i, int val) {        // 单点增加（1-indexed）
        for (; i <= n; i += i & -i) tree[i] += val;
    }

    int sum(int i) {                  // 前缀和 [1, i]
        int s = 0;
        for (; i > 0; i &= i - 1) s += tree[i];
        return s;
    }

    int rangeSum(int l, int r) {      // 区间和 [l, r]
        return sum(r) - sum(l - 1);
    }
};
```

易错点：树状数组下标从 1 开始，原数组对应下标需 +1。`i & -i` 取最低位 1。

19.2 线段树（区间和 + 懒标记）

```
struct SegTree {
    int n;
    vector<long long> tree, lazy;
    SegTree(const vector<int>& a) {
        n = a.size();
        tree.resize(4 * n);
        lazy.resize(4 * n, 0);
        build(a, 1, 0, n - 1);
    }

    void build(const vector<int>& a, int node, int l, int r) {
        if (l == r) { tree[node] = a[l]; return; }
        int m = (l + r) / 2;
        build(a, node * 2, l, m);
        build(a, node * 2 + 1, m + 1, r);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }

    void push(int node, int l, int r) {
        if (lazy[node]) {
            int m = (l + r) / 2;
            tree[node * 2] += lazy[node] * (m - l + 1);
            tree[node * 2 + 1] += lazy[node] * (r - m);
            lazy[node * 2] += lazy[node];
            lazy[node * 2 + 1] += lazy[node];
            lazy[node] = 0;
        }
    }

    void add(int ql, int qr, int val, int node, int l, int r) {
        if (ql <= l && r <= qr) {
            tree[node] += 1LL * val * (r - l + 1);
            lazy[node] += val;
            return;
        }
        push(node, l, r);
        int m = (l + r) / 2;
        if (ql <= m) add(ql, qr, val, node * 2, l, m);
        if (qr > m)  add(ql, qr, val, node * 2 + 1, m + 1, r);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }

    long long query(int ql, int qr, int node, int l, int r) {
        if (ql <= l && r <= qr) return tree[node];
        push(node, l, r);
        int m = (l + r) / 2;
        long long res = 0;
        if (ql <= m) res += query(ql, qr, node * 2, l, m);
        if (qr > m)  res += query(ql, qr, node * 2 + 1, m + 1, r);
        return res;
    }
```

```
    void add(int l, int r, int v) { add(l, r, v, 1, 0, n - 1); }
    long long query(int l, int r) { return query(l, r, 1, 0, n - 1); }
};
```

易错点：
1. 数组开 4 倍空间
2. `push` 在递归访问子节点前调用
3. 区间更新时懒标记累加，不要覆盖

二十、拓扑排序与最小生成树

20.1 拓扑排序（Kahn 算法）

```
vector<int> topoSort(int n, vector<vector<int>>& g, vector<int>& inDeg) {
    queue<int> q;
    for (int i = 0; i < n; i++)
        if (inDeg[i] == 0) q.push(i);

    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : g[u])
            if (--inDeg[v] == 0) q.push(v);
    }
    // 若 order.size() != n，说明存在环
    return order;
}
```

易错点：
1. 记得统计入度：`inDeg[v]++`
2. 建图方向：`a → b` 表示 a 依赖 b，则 `g[b].push_back(a)`，`inDeg[a]++`
3. 判环：结果长度 ≠ n

20.2 最小生成树——Kruskal

```
struct Edge { int u, v, w; };

int find(vector<int>& p, int x) {
    return p[x] != x ? p[x] = find(p, p[x]) : x;
}

int kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end(), [](auto& a, auto& b) { return a.w < b.w; });
    vector<int> p(n);
    iota(p.begin(), p.end(), 0);
    int ans = 0, cnt = 0;
    for (auto& e : edges) {
        int ru = find(p, e.u), rv = find(p, e.v);
        if (ru != rv) {
            p[ru] = rv;
            ans += e.w;
            if (++cnt == n - 1) break;
        }
    }
    return cnt == n - 1 ? ans : -1; // -1 表示不连通
}
```

易错点：

1. 边按权重**从小到大**排序
2. 用并查集判环
3. 记录已选边数 cnt，提前 break

20.3 最小生成树——Prim

```
int prim(int n, vector<vector<pair<int,int>>>& g) {
    vector<int> dist(n, INT_MAX), vis(n, 0);
    dist[0] = 0;
    priority_queue<pair<int,int>, vector<...>, greater<>> pq;
    pq.push({0, 0});
    int ans = 0, cnt = 0;
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (vis[u]) continue;
        vis[u] = 1;
        ans += d;
        if (++cnt == n) break;
        for (auto [v, w] : g[u])
            if (!vis[v] && w < dist[v])
                pq.push({dist[v] = w, v});
    }
    return cnt == n ? ans : -1;
}
```

二十一、数位 DP

21.1 模板

```
// 统计 [0, n] 中满足条件的数字个数
// 参数：pos（当前位）, mask（状态）, limit（是否受上限约束）, lead（前导零）
function<long long(int, int, bool, bool)> dfs = [&](int pos, int mask, bool limit, bool lead)
-> long long {
    if (pos < 0) return 1;  // 根据题目需求修改返回值
    if (!limit && !lead && memo[pos][mask] != -1) return memo[pos][mask];

    int up = limit ? s[pos] - '0' : 9;
    long long res = 0;
    for (int d = 0; d <= up; d++) {
        if (/* 不合法条件 */) continue;
        res += dfs(pos - 1, newMask, limit && d == up, lead && d == 0);
    }
    if (!limit && !lead) memo[pos][mask] = res;
    return res;
};

// 调用：dfs(len-1, 0, true, true)
```

思路：记忆化搜索，将数字按位分解，逐位枚举。
易错点：
1. `limit` 为 true 时不要存 memo（受上限约束的状态不通用）
2. `lead` 处理前导零对状态的影响
3. 数字用字符串存储，`pos` 从最高位开始

21.2 例题框架

```
// 求 [L, R] 中满足条件的个数 → calc(R) - calc(L-1)
long long calc(int x) {
    string s = to_string(x);
    int n = s.size();
    vector memo(n, vector<long long>(状态数, -1));
    // 定义 dfs...
    return dfs(n-1, 初始状态, true, true);
}
```

二十二、灵神模板补充——高性能 IO 与杂项

22.1 快速输入（竞赛必备）

```
// 读整数（比 cin 快 10 倍）
int read() {
    int x = 0, f = 1; char c = getchar();
    while (!isdigit(c)) { if (c == '-') f = -1; c = getchar(); }
    while (isdigit(c)) { x = x * 10 + c - '0'; c = getchar(); }
    return x * f;
}

// 读 long long
long long readLL() {
    long long x = 0, f = 1; char c = getchar();
    while (!isdigit(c)) { if (c == '-') f = -1; c = getchar(); }
    while (isdigit(c)) { x = x * 10 + c - '0'; c = getchar(); }
    return x * f;
}
```

易错点： `getchar()` 会读入换行和空格，注意处理多组数据的空行。

22.2 分组循环（灵神思路）

```
// 适用于将数组分成若干连续段，对每段单独处理
int i = 0;
while (i < n) {
    int start = i;
    while (i < n && /* 属于同一组的条件 */) i++;
    // 处理 [start, i-1] 这段
}
```

应用： 连续相同数字分组、连续递增/递减分组等。

22.3 二分答案框架补充

```
// 最大化最小值
while (l < r) {
    int mid = (l + r + 1) / 2;  // 注意 +1 避免死循环
    if (check(mid)) l = mid;
    else r = mid - 1;
}

// 最小化最大值
while (l < r) {
    int mid = (l + r) / 2;
    if (check(mid)) r = mid;
    else l = mid + 1;
}
```

易错点：「最大化最小值」用 `mid = (l+r+1)/2`，防止死循环。

附录：竞赛易错点速查

🚨 常见低级错误

错误	正确做法
忘记初始化	数组/cnt/dp 等马上 `memset` 或 `= {0}`
数组越界	访问 `nums[i-1]` 时先检查 `i>0`
二分死循环	`mid = l + (r-l)/2`（不会死循环）
BFS 忘记标记	入队时立即标记已访问
int 溢出	`(long long)` 强转，或用 `long long`
字符串末尾缺 `'\0'`	C 字符串手动加 `'\0'`
模负数	`(x % k + k) % k`
DFS 栈溢出	层数很深时用迭代/BFS
排序比较函数	返回 `a-b` 可能溢出，用 `(a>b)-(a<b)`
erase 后下标	`erase` 后数组长度变了，注意 `i--`

🎯 竞赛时间分配建议

读题 & 分析 (20%) → 编码 (50%) → 调试 (30%)

- 前 10 分钟通读所有题目，标记难度
- 优先做有把握的题（保底分）
- 卡壳超过 30 分钟果断换题
- 最后 15 分钟只做检查：边界、溢出、初始化

> **最后叮嘱**：码蹄杯国赛考察的是「思路 + 代码准确性」，与其追求偏难怪题，不如把基础算法的模板和易错点吃透。161 道题中 80% 是基础算法，稳扎稳打即可。

手册基于 161 道 LeetCode 题目整理，生成于 2026.07